BRICKPHP · INTERNALS

Patch Mode

What the server does on every interaction after the page exists: render the screen twice, compare the two renders, and ship the difference. This document covers the patch path only — the initial GET, which sends whole HTML and never diffs, is out of scope. It covers what separates a component from a DOM node, which lifecycle hooks fire in which phase, and how a component finds its state again.

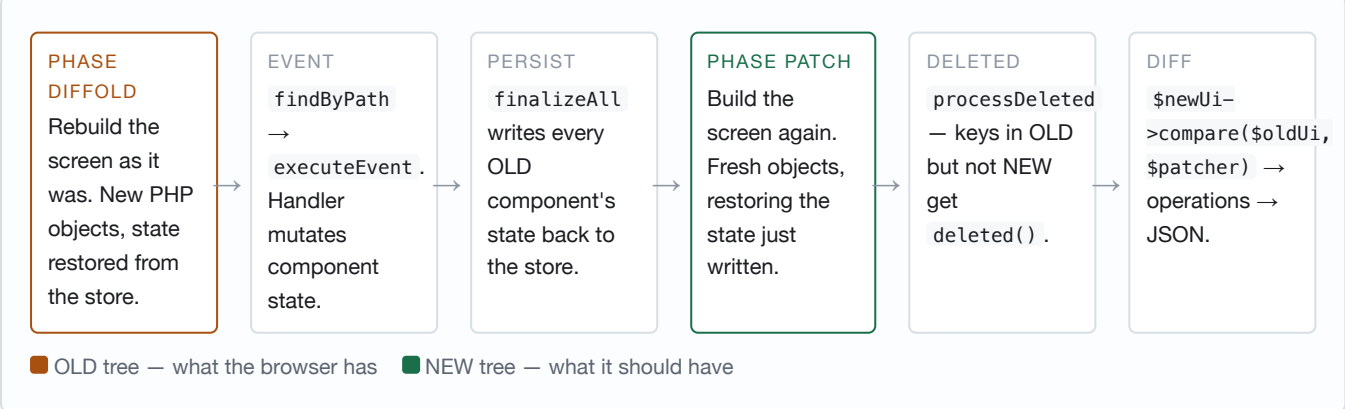
1 The idea in one paragraph

There is no component state on the client and no re-render on the client. When the user clicks something, the browser posts the path of the node that was clicked and the name of the event. The server rebuilds the screen *as it was* — that is the OLD tree — runs the event handler against it, then builds the screen again from the mutated state — the NEW tree. Two DOM trees now sit in memory in the same PHP process, one describing what the browser is showing and one describing what it should show. Comparing them produces a list of small operations (`set_attribute`, `replace_text`, `insert_at`…), each addressed by a path of child indices. That list is the response. The client walks each path and applies each operation. The server holds the truth; the browser holds a projection of it.

> **THE LOAD-BEARING ASSUMPTION**
>
> OLD must be an accurate reconstruction of what is on the screen. Everything else follows from that. The framework spends real effort protecting it — a state hash sent with every request, a `previousPath` override so OLD renders for the URL the user actually has, and force-patch escape hatches (§8) for values the server cannot know have drifted.

2 The three phases of a patch request

`RenderPhase` has three cases, and two of them belong to this request. `Initial` is the GET. `DiffOld` and `Patch` are the two renders of a POST, and every lifecycle decision in the framework keys off which one is running.



```php
// Brick.php — the patch path, condensed to its skeleton
$oldApp = new ($entry::class)();
$oldUi  = $oldApp->render($state, null, RenderPhase::DiffOld);   // 1. replay

$oldUi->hydrateBindings($bindings);                              // 2. re-apply typed-in values
$node = $oldUi->findByPath($path);
$node?->executeEvent($event, $state, $value);                     // 3. run the handler

Component::finalizeAll($state);                                  // 4. persist mutations

$newApp = new ($entry::class)();
$newUi  = $newApp->render($state, null, RenderPhase::Patch);     // 5. rebuild

Component::processDeleted();                                     // 6. deleted() + drop bound state

$patcher = new Patcher();
$newUi->compare($oldUi, $patcher);                                // 7. diff NEW against OLD
$patches = $patcher->getOperations();                             // 8. serialise (DomNode → html)
```

src/Brick.php — the real code adds timing marks, hash checks and error recovery around these eight steps.

Two details in that skeleton are easy to miss and explain a lot of behaviour later. First, **OLD and NEW are different PHP objects**. A component instance never survives from one to the other; only its *state* does, and only by being written to the store in step 4 and read back in step 5. Second, **the event runs against OLD**, which is why the handler closure you captured in `build()` during the OLD render is the one that fires — and why its mutations have to be swept out of every OLD component, not just the one that owned the event.

> **WHY FINALIZEALL, NOT FINALIZE**
>
> A handler can mutate a component *above* the one that owns the event — a row's delete button typically captures `$this` from the list that built it. Finalizing only the event's owner would silently drop that mutation, so the framework finalizes every component registered during the OLD render.

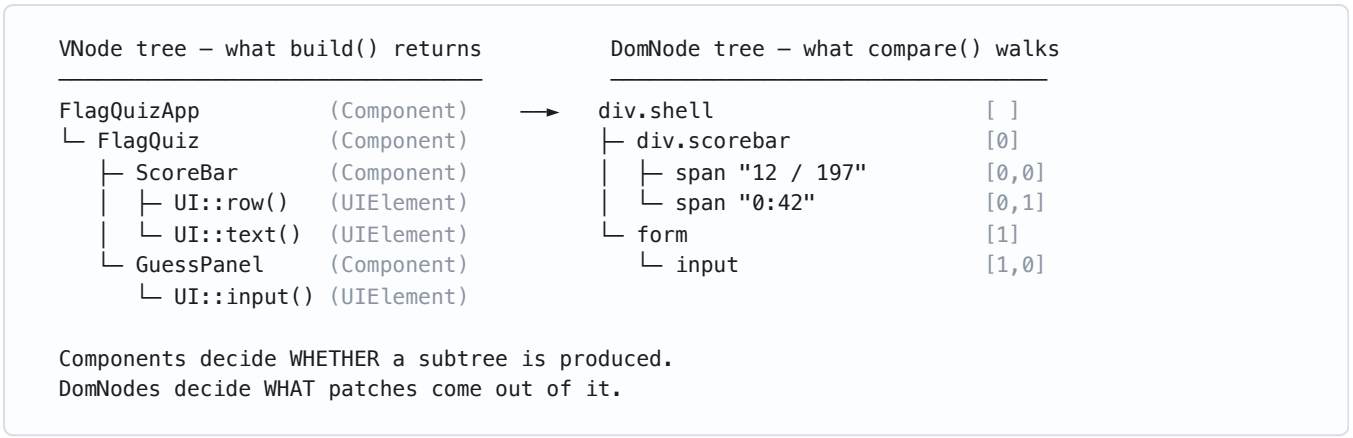
---

3 Two trees: components vs DOM nodes

BrickPHP has two node families and they do different jobs. Conflating them is the single biggest source of confusion about what the diff will do.

- **`VNode`** — the tree you write. `Component`, `StatelessComponent`, and every `UIElement` (`UI::column()`, `UI::text()` …). A VNode has a `render()` and a lifecycle. It is a *producer*.
- **`DomNode`** — the tree that gets compared. `TagDomNode` (an element), `TextDomNode`, `ListDomNode`, `CommentDomNode`, `NoOpDomNode`. A DomNode has `compare()`, `toHtml()` and a path. It is the *subject*.

Rendering collapses the first into the second. Components leave no trace in the DomNode tree: they contribute their build output and disappear. **The diff never sees a component.** It walks elements and text.



	COMPONENT (VNODE)	DOM NODE
Exists in the diff	No — collapses during render	Yes — `compare()` is defined on it
Identity	`path:ClassName` (§5)	Position among siblings, or `key` (§7)
Holds state	Yes, via `useState()`	No
Lifecycle hooks	Yes (§4)	None
Can skip work	`shouldRender()` → whole subtree not built	`frozen` → subtree not compared
Can force work	—	`invalidated` → patch emitted regardless
Unit of reuse	Code	DOM

The practical consequence: *moving a component in the tree does not move its DOM*, and *keying a DOM node does not key the component that produced it*. They are addressed by two independent schemes, described in §5 and §7 respectively.

---

4 Component lifecycle in patch mode

`Component::render()` runs the same sequence in both phases; what differs is which branch each decision takes. Reading it top to bottom:

```php
// src/VNode/Component.php::render() — order of operations
$this->ensureInitialized();          // initialize() once per instance → registers state refs
$saved = $resolved->getState($key);  // look up path:Class in the store
$this->setState($saved);             // write restored values back through the refs
$this->stateSnapshot = …;            // serialise state NOW, before any handler can touch it

$phase === Patch
    ? self::$newRegistry[$key] = $this
    : self::$oldRegistry[$key] = $this;

$this->restored();                   // hook: state is in place

$first = match ($phase) {            // "is this the component's first appearance?"
    Initial => true,
    Patch   => !isset(self::$oldRegistry[$key]),
    default => false,           // DiffOld is a replay — never "first"
};
$first ? $this->created() : $this->updated();

if ($phase === Patch && $old !== null && !$this->shouldRender($old)) {
    return new NoOpDomNode();       // no build, no diff, DOM left alone
}

$child = $this->build();
$rendered = $child->render($state, $this, $phase);
```

Which hook fires when

HOOK	DECLARED ON	DIFFOLD	PATCH	WHAT IT IS FOR
`initialize()`	Component	**once**	**once**	Register state with `useState()`. Once per *instance* — and each phase builds fresh instances, so it runs twice per request.
`restored()`	Component	**always**	**always**	State has been read back out of the store and is live on the properties.
`created()`	BaseComponent	never	**new keys only**	This `path:Class` was not in the OLD tree — a genuinely new component. DiffOld is a replay of something that already exists, so it never fires there.
`updated()`	Component	**always**	**existing keys**	The complement of `created()`: the component was already there.
`shouldRender()`	BaseComponent	not consulted	**always**	Memoisation. `false` returns a `NoOpDomNode`: no `build()`, no comparison, no patches for the subtree.
`build()`	Component	**always**	**unless skipped**	Produce the subtree. Runs in both phases — that is what makes two trees to compare.
`rendered()`	BaseComponent	skipped	**stateless only**	Fires after the subtree exists. See the caveat below — `Component` does not fire it.
`deleted()`	Component	**after both renders**		Key present in OLD, absent from NEW. Fired by `processDeleted()`, which then drops the component's bound state.

HOOK	DECLARED ON	DIFFOLD	PATCH	WHAT IT IS FOR
`finalize()`	Component	via finalizeAll	—	Writes the component's state to the store. Called on OLD instances after the event handler has run.

RENDERED() AND COMPONENT

`rendered()` is declared on `BaseComponent` and called from `BaseComponent::render()`. `Component` overrides `render()` wholesale and does not call it, so the hook fires for `StatelessComponent` subclasses and not for `Component` subclasses. If you need a post-build side effect on a stateful component, put it at the end of `build()` or use `created()` / `updated()`.

Observed: one patch cycle, hook by hook

A three-child screen — a stateful `Leaf A`, a `StatelessComponent`, and a second `Leaf B`. The event bumps A's counter and drops B from the tree.

RECORDED FROM A LIVE RENDER, NOT RECONSTRUCTED

```
--- DIFF OLD ---                          // phase 1: rebuild what the screen shows
A initialize
A restored(count=0) key=0:Leaf
A updated                                 // not created — DiffOld is a replay
B initialize
B restored(count=0) key=2:Leaf
B updated

--- EVENT: A bumps its count, B leaves the tree ---
state keys after finalizeAll: 0:Leaf , 2:Leaf

--- PATCH ---                              // phase 2: rebuild from the mutated state
A initialize                              // a NEW instance: initialize runs again
A restored(count=1) key=0:Leaf            // the mutation arrived through the store
A updated                                 // key was in OLD → updated, not created
Plain rendered                            // stateless component: rendered() does fire

--- PROCESS DELETED ---
B deleted                                 // in OLD, absent from NEW
state keys after processDeleted: 0:Leaf   // B's bound state dropped with it

--- DIFF ---
patch: delete_node path=[2]
patch: replace_text path=[0,0] A:1        // the only textual change in the whole tree
```

Note what the diff did *not* emit: nothing for the untouched stateless component, nothing for the container. Two operations for a tree of six nodes.

5 State keys

A component's state has to survive the gap between two renders of two different object graphs. The bridge is a string key. `VNode::getStateKey()` builds it:

```
protected function getStateKey(): string
{
    $className = static::class;
    $pathStr   = implode('.', $this->path);
    return $pathStr === '' ? $className : "$pathStr:$className";
}
```

So the key is **position in the tree, plus the class name** — `0.1.2:Samples\FlagQuiz\Components\ScoreBar`. The root App, having no path, keys on its class name alone.

Where the path comes from

Not from the DOM. Paths are handed down during render, in `UIElementContent::render()`, as each element walks the children it was given:

```
foreach ($this->children as $child) {
    if ($child instanceof VNode) {
        $child->setPath([...$this->path, $index]);        // ← the component's address
        $domChildren[] = $child->render($state, $this, $phase);
    } elseif (…) { … }
    $index++;                                             // every slot counts, strings included
}
```

Two consequences worth internalising. A component's key is a function of *where it sits in its parent's `content()` call*, and the index advances for every child slot — including plain strings and raw DOM nodes, which are not components at all. And because the class name is part of the key, two different component classes at the same position never collide, while two instances of the same class at different positions never share.

> **OBSERVED: STATE DOES NOT FOLLOW A MOVED COMPONENT**
>
> ```
> counter key (no spacer):    0:Counter -> n=7        // state saved under 0:Counter
> counter key (spacer added): 1:Counter -> n=0        // same component, new address
>                                                     // ← state did NOT follow
> ```

Inserting one sibling above a component renames its key, and the lookup in the store misses. From the framework's point of view this is not a move at all: the old key is in OLD but not in NEW, so it is *deleted* (and its bound state discarded), while the new key is in NEW but not OLD, so it is *created*. If a subtree's position can shift, keep the state above the shifting boundary — in a parent whose own position is stable — or make it global.

Reading, writing, and dropping

STEP	CODE	EFFECT
Register	`useState($this->n)`	Binds the property *by reference* into a `StateRef`, so restore writes straight through to it. Call it in `initialize()`.
Restore	`getState($key)`	An indexed array, applied in registration order. A count mismatch aborts the restore rather than corrupting it.
Persist	`finalize($state)`	`saveState($key, $values)`, run for every OLD component after the handler.
Drop	`processDeleted()`	`removeState($key)` — but only if the component registered at least one `Bound` ref.
Share	`useGlobalState($ref, 'cart')`	A fixed key of your choosing. Position-independent, survives deletion, shared by every component that names it.

`StateLifecycle::Bound` (the default) means the state belongs to the component and dies with it. `Unbound` means the entry outlives the component, so a screen the user navigates away from and back to finds its values again. The choice is per-registration; a single `Bound` ref is enough to make the component's whole entry removable.

The staleness guard

Every response carries `sha1(serialize($state->getAll()))`, and the client echoes it on the next request. If the store hashes differently than the client expects — another tab moved the state, or a deploy changed its shape — the server does not attempt a patch. It answers `{success: false, reload: true}`, because OLD could no longer be reconstructed faithfully, and every patch downstream of a wrong OLD is wrong too.

---

6 How two DOM nodes are compared

`compare()` is called on the NEW node with the OLD node as its argument — `$newUi->compare($oldUi, $patcher)`. Read every method as "make the browser look like `$this`". `TagDomNode::compare()` checks in this order:

#	CHECK	OUTCOME
1	`$this->frozen`	Return immediately. No attributes, no children, no patches — the subtree is not even walked.
2	Other is not a tag, or tags differ	`replace_node` with this node's full HTML. The old subtree's listeners are unbound, the new subtree's are bound.
3	`$this->invalidated`	Propagate the flag to every child, so the whole subtree force-emits.
4	Attributes	`set_attribute` for each attribute that is new, changed, or force-flagged; `remove_attribute` for each that vanished.
5	Classes	Compared as a set of names; emitted as one `set_attribute class="…"` when they differ.
6	Listeners	`diffEvents()` — appeared listeners bind, vanished ones unbind, unchanged ones keep their registration.
7	Children	Keyed or positional (§7).

The leaf cases are smaller. `TextDomNode::compare()` emits `replace_text` when the content differs (or the node is invalidated) and `replace_node` when OLD was not text at all. `NoOpDomNode::compare()` is an empty method — that is the whole mechanism behind `shouldRender() === false`: the component returns a node whose comparison does nothing, so the browser keeps whatever it had.

> **OBSERVED: SHOULDRENDER()**
>
> ```
> same props   -> Memo::build() calls: 0, patches: none
> prop changed -> Memo::build() calls: 1, patches: replace_text[1,0] "changed"
> ```

The decision is taken *per level, while descending* — there is no pass that builds the whole NEW tree and then decides what to keep. `shouldRender()` is consulted before `build()`, so a `false` stops the traversal at that node and nothing below it is constructed at all.

> **MEMOISING A PARENT DELETES ITS CHILDREN'S STATE**
>
> `frozen` and `shouldRender()` skip in *different* phases, and the difference has teeth. A frozen subtree is skipped in DiffOld *and* Patch, so the components inside it are missing from both registries and nothing happens to them. A `shouldRender()` of `false` skips the NEW render only — the OLD replay already walked that subtree and registered every component in it. `processDeleted()` compares the two registries, finds those descendants in OLD and not in NEW, and treats them as deleted: `deleted()` fires and their bound state is dropped from the store. Observed, for a stateful grandchild under a parent that skipped:
>
> ```
> parent rebuilds -> Grandchild restored(n=42) … stored keys now: 0.0.0:Grandchild
> parent skips    -> Grandchild deleted()      … stored keys now: (none)
> ```
>
> Which is the deeper reason `shouldRender()` belongs on leaves: on a parent it does not merely risk missing a change, it destroys the state underneath it.

`hasChanged($old)` makes that decision cheap to implement: at the top of `render()` each component serialises its own declared properties into a snapshot string, *before* any handler can run, and the two snapshots are compared. It is a leaf-component tool. Returning `false` from a parent skips its descendants' builds too, and a child's own state can change without the parent's snapshot moving at all — the change would simply never reach the DOM.

7 Children: positional and keyed

If *every* child on both sides carries a key, the keyed strategy runs; otherwise the positional one does. There is no partial mode.

Positional compares child *i* with child *i*, deletes the tail when NEW is shorter, and inserts when it is longer. It is correct and it is cheap, but it has no concept of identity: removing an item from the middle of a list rewrites every item after it.

Keyed matches by key instead. Each key maps to a FIFO queue of old indices, and a candidate is accepted only if its old index is strictly greater than the last one matched. Anything that would have to move backwards is deleted and re-inserted instead — the diff produces correct order without emitting move operations, at the cost of losing DOM state (focus, scroll position, a playing animation) for the nodes that get re-created.

> **OBSERVED: THREE CHILDREN, THE MIDDLE ONE REMOVED**
>
> ```
> keyed     -> remove_at[]#1                              // one op; a and c are untouched
> unkeyed   -> delete_node[2], replace_text[1,0] "c"      // b is rewritten into c, tail dropped
> ```

Keys are also usable as a deliberate *replace* signal. Keying a node on its own value — `->key('right-' . $count)` — guarantees that a changed value produces a new node rather than a patched one, which restarts CSS animations that only run on element creation.

8 Escape hatches

The diff is a comparison of two server renders, so it detects exactly the changes the server can see. Four flags exist for the cases where that is either too much work or not enough.

FLAG	DIRECTION	WHEN
`frozen`	Compare less	Genuinely static UI — a logo, a footer. The subtree is skipped in both renders and never compared — which is why, unlike `shouldRender()`, it leaves the state of any component inside it alone.
`invalidate()`	Compare more	Force-patch this node and everything under it. For subtrees whose correctness the server cannot verify.
`invalidateAttr(name)`	Compare more	One attribute, force-patched; the rest of the node diffs normally.
`invalidateText()`	Compare more	The direct text children, force-patched.

The "compare more" flags all exist for one situation: **the DOM has drifted from what OLD says it is**. Two server renders agreeing tells you nothing if something other than the server has been writing to that node.

> **A WORKED EXAMPLE**
>
> The Vexi quiz shows elapsed time. The server renders the seconds it counted, and a client script ticks the display between requests. By the time the next request arrives, the DOM says `0:27` while both OLD and NEW say `0:26` — the server's two renders agree, so an honest diff emits nothing, and the clock would jam at whatever the script last wrote. `invalidateText()` on that cell forces the text out on every patch, and the display re-anchors to the server's figure. The same reasoning covers a value the user typed, a canvas a chart library drew into, and anything derived from `time()` or randomness.

9 The patch protocol

`Patcher` is a plain collector — the diff calls methods on it, and `getOperations()` serialises any `DomNode` it is holding into an `html` string. The result is a flat array in emission order.

OPERATION	PAYLOAD	CLIENT ACTION
`replace_node`	path, html	Clean up listeners, `replaceWith` the parsed element.
`insert_node`	path, html	Insert before `parent.children[i]`, or append past the end.
`delete_node`	path	Clean up listeners and remove.
`replace_text`	path, text	Assign `textContent`.
`set_attribute`	path, name, value	`setAttribute` — plus the matching *property* for `value` and `checked`, since a form control's attribute is only its default.
`remove_attribute`	path, name	`removeAttribute`.
`insert_at / remove_at / update_at`	parent path, index, html	Keyed-list operations, addressed by parent plus index rather than by full child path.

```
// The client resolves every path the same way the server built it:
function findNodeByPath(path) {
    let node = document.body.firstElementChild;   // the app root
    for (const index of path) node = node.children[index];
    return node;
}
```

Element children only — which is why `UI::text()` renders as a `<span>` rather than a bare text node. A raw text node would not occupy an index in `children`, and every path after it would point at the wrong element. Giving text an element wrapper keeps server paths and client paths in step.

10 Source map

FILE	RESPONSIBILITY IN PATCH MODE
`src/Brick.php`	The POST path: hash check, DiffOld render, event dispatch, finalizeAll, Patch render, processDeleted, compare, response.
`src/VNode/RenderPhase.php`	The three-case enum every phase decision reads.
`src/VNode/Component.php`	State registration and restore, the registries, restored / created / updated / deleted, finalizeAll, processDeleted.
`src/VNode/BaseComponent.php`	The minimal component surface: build, created, shouldRender, rendered.
`src/VNode/Patcher.php`	Operation collection and serialisation.
`src/UI/DomNode.php`	Paths, keys, the frozen / invalidated flags, the compare contract.
`src/UI/TagDomNode.php`	The element diff: attributes, classes, events, positional and keyed children.
`src/UI/TextDomNode.php`	Text comparison.

FILE	RESPONSIBILITY IN PATCH MODE
`src/UI/UIElementContent.php`	Child path assignment — where a component's state key is decided.
`src/State/StateManager.php`	getState / saveState / removeState, keyed by `path:Class`.
`assets/brick.js`	Path resolution and patch application on the client.

BrickPHP internals · Patch mode. Companion to *How the DOM Diff Works*, which covers the initial render and the addressing scheme in full. Behavioural claims marked *observed* were recorded by driving the render phases directly against the framework.